

Assumptions per Entity

dim_date

- Represents a calendar date that is derived from FL_DATE
- date_id is a key with the format YYYYMMDD
 - Will aid in joins and grouping
- One date can be referenced by many flights

dim_airport

- Represents an airport from T_MASTER_CORD and is identified by airport_id
- Airport location (latitude and longitude) and other geographic info will be here
- airport_wac will map the airport to a WAC region
 - For future reference WAC = World Area Code according to the Bureau of Transportation

dim_carrier

- Represents an airline “carrier” from T_CARRIER_DECODE and is identified by airline_id
- Carrier names, region, and group will be the attributes that describe each airline
 - Shouldn’t repeat per flight

dim_wac

- Represents the WAC geography codes from T_WAC_COUNTRY_STATE
- Used by airports and airline carriers and stored for flights to quickly distinguish flights by region

dim_delay_type

- Define delay categories
 - Carrier, Weather, NAS, Security, Late Aircraft

fact_flight

- Grain: one row per flight instance from T_ONTIME_MARKETING
 - 600,000 entires
- Should contain only foreign keys, measures, and flags

fact_flight_delay

- This acts as a bridge
 - Each row represents the flight and delay_type
 - Constraint: delay_minutes >0
 - Allows for many-to-many relationships between flights and delay types

saved_preset

- Represents a user-saved filter configuration
- Filters_json stores the full filter parameters as a JSON string, which can be read and generated into a query when loading the preset
- created_at and updated_at track when the preset was created and last modified

Relationships and Cardinalities

dim_date **(1)** -> **(many)** fact_flight

Each flight occurs at exactly one calendar date, while a single date can have many flights.

dim_carrier **(1)** -> **(many)** fact_flight

Each flight is marketed by one carrier, while a single carrier can have many flights

dim_airport **(1)** -> **(many)** fact_flight [origin]

Each flight has exactly one origin airport, while an airport can be an origin airport for many other flights.

dim_airport **(1)** -> **(many)** fact_flight [destination]

Each flight has exactly one destination airport, while an airport can be a destination airport for many other flights.

dim_wac **(1)** -> **(many)** dim_airport

Each airport belongs to one WAC region, while a WAC region can have many airports.

dim_wac **(1)** -> **(many)** dim_carrier

Each carrier is headquartered/registered in one WAC region, while a WAC region can have multiple carriers registered within it.

fact_flight **(many)** <-> **(many)** dim_delay_type

A flight may have either zero or multiple causes for delay, and each delay type can apply to many flights

saved_preset **(standalone)**

saved_preset has no foreign key relationships to other entities. It exists as an independent store for user filter configurations for the dashboard. This does mean presets are globally accessible to all users of the application since this is designed as a single-instance analysis tool. All presets are shared and any user can create, view, edit, or delete them.

Normalization

The original flight dataset we use contains repeated information about various airports, carriers, geographic regions, and dates for each flight row. Therefore, in order to eliminate any redundancy we went ahead and, in our schema, transformed the data into a fact table (*fact_flight*) which only contains keys and measures while having other multiple dimension tables (*dim_airport*, *dim_carrier*, *dim_wac*, *dim_date*) which contain the descriptive attributes that depend on their primary keys. The delay causes are normalized further into *dim_delay_type* with the bridge table *fact_flight_delay* to aid in the many-to-many relationship between flights and delay types. In each table, all the non-key attributes depend only on the primary key and not on the other non-key attributes allowing for our schema to fall under the 3NF normalization.

Relational Schema

```
/*
  Datatypes:
    INT: integer or flag (0,1)
    BIGINT: larger whole number for bigger values than INT
    VARCHAR(n): essentially strings with length n
    DECIMAL(p,s): essentially like a double
    DATE: calendar date (year-month-day)
    TIMESTAMP: metadata (created_at, updated_at)
    TEXT: JSON for any configs present or string text of any
           length
*/

dim_date(
  date_id: INT [PK],
  full_date: DATE, year: INT,
  quarter: INT, month: INT,
  day_of_month: INT,
  day_of_week: INT)

dim_wac(
  wac: INT [PK],
  wac_name: VARCHAR(100),
  world_area_name: VARCHAR(100),
  country_short_name: VARCHAR(100),
  country_code_iso: VARCHAR(10),
  state_code: VARCHAR(10),
  state_name: VARCHAR(100),
  state_fips: VARCHAR(10),
  is_latest: INT)
```

```
dim_airport(  
    airport_id: INT [PK],  
    airport: VARCHAR(10),  
    display_airport_name: VARCHAR(200),  
    display_airport_city_name_full: VARCHAR(200),  
    airport_state_code: VARCHAR(10),  
    airport_state_name: VARCHAR(100),  
    airport_state_fips: VARCHAR(10),  
    airport_country_name: VARCHAR(100),  
    airport_country_code_iso: VARCHAR(10),  
    airport_wac: INT [FK to dim_wac.wac],  
    latitude: DECIMAL(10,6),  
    longitude: DECIMAL(10,6),  
    airport_is_latest: INT)  
  
dim_carrier(  
    airline_id: INT [PK],  
    carrier: VARCHAR(10),  
    carrier_name: VARCHAR(200),  
    unique_carrier: VARCHAR(10),  
    unique_carrier_name: VARCHAR(200),  
    wac: INT [FK to dim_wac.wac],  
    carrier_group: VARCHAR(50),  
    region: VARCHAR(50))  
  
dim_delay_type(  
    delay_type_id: INT [PK],  
    delay_type_name: VARCHAR(50))  
  
fact_flight(  
    flight_id: BIGINT [PK],  
    date_id: INT [FK to dim_date.date_id],  
    mkt_carrier_airline_id: INT [FK to dim_carrier.airline_id],  
    mkt_carrier_fl_num: INT,  
    origin_airport_id: INT [FK to dim_airport.airport_id],  
    dest_airport_id: INT [FK to dim_airport.airport_id],  
    origin_wac: INT [FK to dim_wac.wac],  
    dest_wac: INT [FK to dim_wac.wac],  
    dep_delay: DECIMAL(8,2),  
    arr_delay: DECIMAL(8,2),  
    arr_del15: INT,  
    cancelled: INT,  
    diverted: INT,  
    air_time: DECIMAL(8,2),  
    distance: DECIMAL(10,2))
```

```
fact_flight_delay(  
    flight_id: BIGINT [PK, FK to fact_flight.flight_id],  
    delay_type_id: INT [PK, FK to dim_delay_type.delay_type_id],  
    delay_minutes: DECIMAL(10,2))  
  
saved_preset(  
    preset_id: BIGSERIAL [PK],  
    preset_name: VARCHAR(200),  
    filters_json: TEXT,  
    created_at: TIMESTAMP,  
    updated_at: TIMESTAMP)
```

